

Facilibook — School Facility Booking System

Documentation · Package: com.main · Backend: Supabase (PostgreSQL) · UI: JavaFX (with a console fallback)

1. What it is

Facilibook is a school facility booking system for Our Lady of Lourdes College. Students, teachers, and admins log in, browse facilities (labs, classrooms, gyms, auditorium, library, conference room), check availability, and book time slots. Admins additionally manage the facility catalog and review/approve bookings.

The project has two parallel front ends sharing the same service layer:

- A JavaFX GUI (FxApp.java, later migrated to FXML screens, see section 5) — this is the actively developed one.
- A console/CLI version (FACILIBOOK.java) — a menu-driven terminal client, useful for quick testing of the service layer without the UI.

Both call into the same three services (UserService, FacilservJRD, BookServ), which talk to a shared Supabase Postgres database.

2. Tech stack

Layer	Technology
Language	Java (records, switch expressions, var-style patterns -> Java 17+)
GUI	JavaFX (Scene Builder-compatible FXML + CSS)
Database	PostgreSQL via Supabase (local dev via Supabase CLI, config.toml)
DB driver	JDBC (org.postgresql.Driver) – raw PreparedStatements, no ORM
Email	Jakarta Mail (SMTP) for OTP password-reset codes
Config	Environment variables, with a local .env file fallback
Data files	facilities.json, bookings.json, users.json appear to be seed/reference snapshots of the DB tables (schema.sql defines the actual source of truth)

3. Architecture

```

com.school
├── FACILIBOOK.java      Console/CLI entry point + menu loop
├── FxLauncher.java      javafx-safe main() wrapper (shaded-jar workaround)
├── FxApp.java           Original JavaFX Application (single-file, style-based
screen switching)
├── model/
│   ├── User.java       id, name, email, password, role (STUDENT/TEACHER/ADMIN)
│   ├── Booking.java    id, userId/Name, facilityId/Name, date, startTime,
endTime, purpose, status
│   └── FacilityKRT.java id, name, type, location, capacity, description, active
├── service/
│   └── DatabaseStorage.java Connection factory, schema bootstrap, .env/env-var
config
│   ├── UserService.java auth, register, CRUD on users
│   ├── FacilservJRD.java facility CRUD, active/soft-delete toggling
│   ├── BookServ.java    booking CRUD, overlap/availability checks
│   ├── OtpService.java  6-digit OTP password-reset flow
│   └── EmailService.java SMTP sender for OTP emails
├── util/
│   ├── SessMgrKRT.java  static in-memory "current logged-in user" session
│   ├── Navigator.java   FXML screen switcher + shared sidebar wiring
│   └── LoadingSpinner.java animated Lucide-style spinner widget
└── controller/         One controller per FXML screen (see section 5)

```

Naming quirk callout: several classes have deliberately "obfuscated-looking" suffixes (FacilservJRD, BookServ, FacilityKRT, SessMgrKRT) — these are just this project's internal naming convention, not generated code.

Two generations of the service layer

The codebase contains two versions each of UserService, BookServ, FacilservJRD, and DatabaseStorage. The newer versions add:

- **DatabaseStorage:** .env file fallback (loadDotenv()), initAsync() (non-blocking startup init), a password_resets table.
- **UserService:** updatePasswordByEmail(), updateUser() (profile editing with email-collision check).
- **BookServ:** bookings now default to PENDING status (not auto-APPROVED) and need admin approval; getBlockingByDate() for the availability screen.
- **FacilservJRD:** soft delete split into explicit enableFacility() / disableFacility(), plus hasBookings() (prevents hard-deleting a facility with booking history) and hardDelete().

Callout: The newer versions are the ones that match the current FXML/controller code (e.g. PendingBookingsController, ManageFacilitiesController) – treat those as canonical going forward and retire the older pair to avoid confusion.

5. Screens/Ui (JavaFX / FXML)

Built from a shared dark-theme sidebar shell (dashboard.css), navigated via `Navigator.go(stage, screenName)`, which loads the matching FXML, wires the shared sidebar buttons, fills in the logged-in user's name/role, and routes through a loading-screen transition on every navigation except login/register.

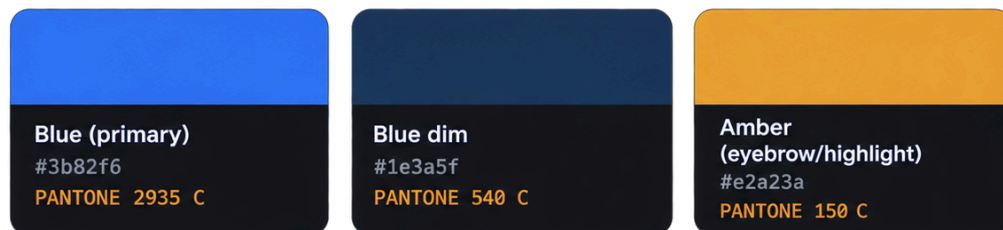
Screen	FXML/Controller	Who sees it	Status
Login	login.fxml / LoginController	everyone	done — includes inline "Forgot Password" OTP reset flow
Register	register.fxml / RegisterController	everyone	done
Dashboard	dashboard.fxml / DashboardController	everyone	done — status bar + stat cards for the user's bookings
Class Schedule	class-schedule.fxml / ClassScheduleController	everyone	done (placeholder: shows the user's own APPROVED bookings, not a real class-schedule model)
Room Availability	availability.fxml / AvailabilityController	everyone	done — live search bar, background-loaded list, click a row to open Room Details
Room Details	room-details.fxml / RoomDetailsController	everyone	done — read-only schedule for one facility
Book Facility	book-facility.fxml / BookFacilityController	everyone	done — 6-step wizard: date, search, room, start time, end time, review/confirm
Cancel Booking	cancel-booking.fxml / CancelBookingController	everyone	done — cancels only PENDING bookings; APPROVED shows

			an "auto-proceeds"
My Profile	my-profile.fxml / MyProfileController	everyo ne	done — edit name/email/passwo rd
Pending Bookings	pending-bookings.fxml / PendingBookingsController	ADMIN	done — Accept/Reject cards
All Bookings	all-bookings.fxml / AllBookingsController	ADMIN	done — status filter tabs, hard-delete
Manage Facilities	manage-facilities.fxml / ManageFacilitiesController	ADMIN	done — Add/Edit/Enable- Disable/hard-Delete

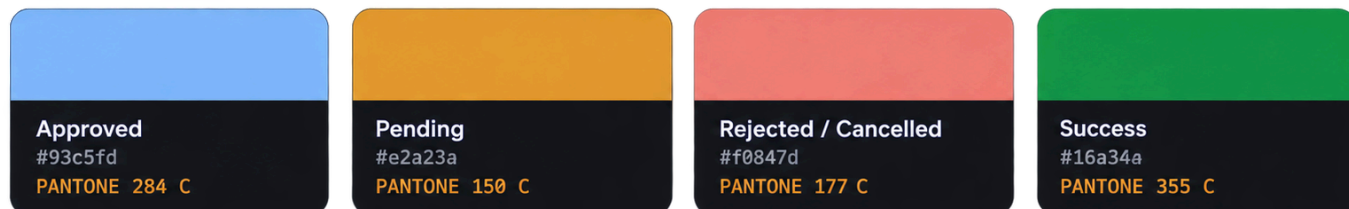
5.2 Ui/UX

Background #0b0c0e PANTONE 419 C	Panel #141519 PANTONE 426 C	Panel 2 / hover #1b1c21 PANTONE 426 C
Line / border #26272c PANTONE 426 C	Text #f2f3f5 PANTONE 656 C	Muted text #8b8d94 PANTONE 444 C

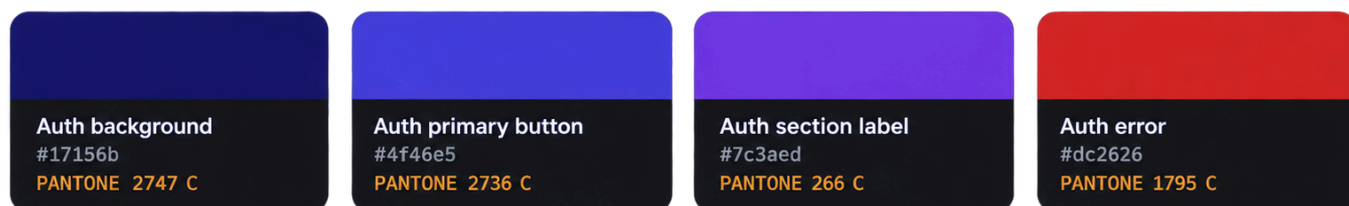
Accents — primary actions, active nav, links

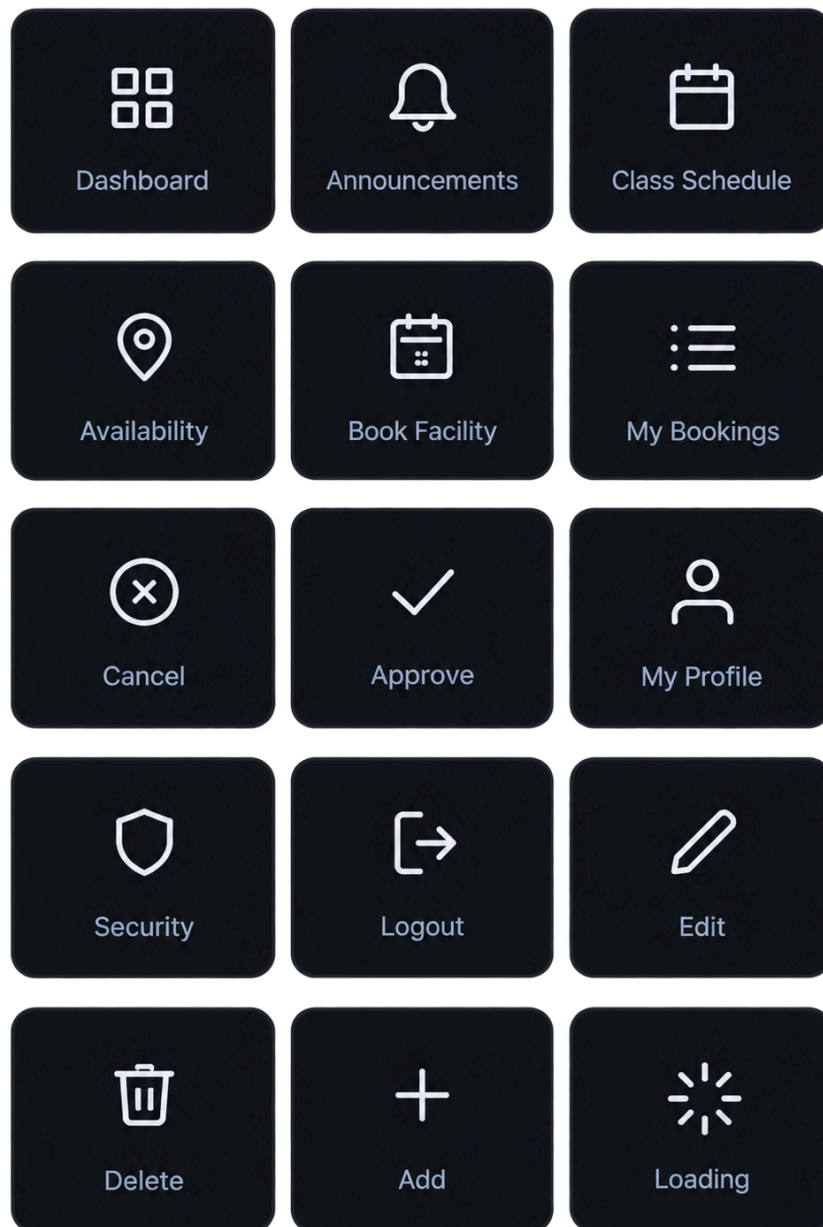


Status — booking states



Login / Register screen (separate light theme)





6. Core flows

6.1 Authentication

`UserService.authenticate(email, password)` — case-insensitive email match, exact password match (see Known Issues re: plaintext). On success, `SessMgrKRT.login(user)` stores the user in a static field for the process lifetime (single-user desktop app, no multi-session concerns).

6.2 Registration

UserService.register(name, email, password, role) — role limited to STUDENT/TEACHER from the UI; rejects if the email already exists.

6.3 Booking creation

BookServ.createBooking(...):

1. isAvailable() checks for time overlap against all non-cancelled/non-rejected bookings for that facility+date.
2. If free, inserts with status PENDING (newer version) — an admin must approve it via Pending Bookings before it's confirmed.
3. Returns empty if the slot is taken (surfaced in the UI as "slot overlaps / already booked").

Availability is computed on the fly by scanning existing bookings — there's no separate "slots" table.

6.4 Booking lifecycle

```
PENDING --(admin Approve)--> APPROVED
PENDING --(admin Reject)---> REJECTED
PENDING --(owner/admin Cancel)--> CANCELLED
APPROVED --(admin hard Delete only, via All Bookings)--> removed
```

Regular users can only cancel their own PENDING bookings from the UI; APPROVED bookings are treated as locked in ("proceeds automatically").

6.5 Password reset (OTP)

OtpService + EmailService:

1. User requests a code — a 6-digit OTP is generated, stored in password_resets with a 10-minute expiry and a 60-second resend cooldown.
2. Email sent via SMTP if SMTP_HOST/SMTP_USER/SMTP_PASS are configured; otherwise falls back to demo mode (code is printed to console / shown in the UI directly).
3. Verify: max 5 attempts before the code is invalidated.
4. On success, UserService.updatePasswordByEmail() sets the new password and the reset row is cleared.

6.6 Facility lifecycle (admin)

- **Soft delete** (default, "Delete" in the older CLI/FxApp code): sets active = false, hidden from booking screens but keeps booking history intact.
- **Enable/Disable** (newer ManageFacilitiesController): same underlying active flag, explicit toggle either direction.

- **Hard delete:** only allowed if `hasBookings()` returns false, otherwise blocked with an error dialog telling the admin to disable instead.

7. Configuration & running it

Environment variables (required)

SUPABASE_DB_URL (JDBC URL, e.g. `jdbc:postgresql://host:5432/postgres`), SUPABASE_DB_USER, SUPABASE_DB_PASS. The newer DatabaseStorage also accepts these from a local `.env` (or `supabase/.env`) file — real environment variables always win over the file.

Optional (email/OTP)

SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM, SMTP_FROM_NAME. Without these, password reset still works in demo mode (code shown on-screen instead of emailed).

Local Supabase (config.toml)

standard supabase init config — Postgres on 54322, Studio on 54323, local inbox on 54324 for catching test emails, API on 54321. `schema.sql` is the canonical DDL + seed script to run against a fresh database (supabase db reset or via the SQL editor).

Running

GUI — run `FxLauncher.main()` (needed instead of `FxApp` directly because JavaFX can't be the main class of a shaded/fat jar). CLI — run `FACILIBOOK.main()`, but note its `main()` currently just forwards to `FxLauncher.main()`, so at present both entry points launch the GUI; to actually run the console menu (`mainMenu()`), that forwarding line needs to be removed/commented.

8. Known issues / gaps

Numbered list, worth triaging before this goes further:

1. Passwords stored in plaintext, both in the users table and `users.json`. No hashing (e.g. BCrypt) anywhere in `UserService`. High priority to fix before any real deployment.
2. `FACILIBOOK.main()` doesn't run the console menu — it just delegates to `FxLauncher.main()`, so the CLI code path (`mainMenu()`, `loginMenu()`, etc.) is currently dead code from the app's actual entry point.
3. Two generations of the service classes are both present in the codebase (see section 3) — should be reconciled into one canonical version to avoid someone editing the stale copy.
4. Class Schedule screen is a placeholder — it repurposes the user's own approved bookings rather than a real class/schedule model.

5. Announcements and Security (standalone change-password) screens are not built yet (per the project's UI-TODO).
6. Availability checks are date-only, not real-time-aware on some screens (isCurrentlyInUse() only checks "today").
7. Hardcoded UUIDs for demo seed rows rely on ON CONFLICT (id) DO NOTHING — reseeding is idempotent, but there's no migration versioning beyond the one schema.sql file.
8. A couple of stray non-demo user rows exist in users.json with a malformed email (kurtnajempoy@email, missing a domain) — likely test data that should be cleaned out before sharing the seed file.
9. Some sidebars are inconsistent: the two admin-only screens' sidebars include an "ADMIN" nav group that earlier-built screens don't yet show — needs the shared-sidebar fix already flagged in the project roadmap.

9. Quick reference — class responsibilities

Class	Responsibility
SessMgrKRT	Holds the single logged-in User for the process; isAdmin()/isLoggedIn() helpers
Navigator	Loads FXML, applies the shared sidebar (hiding admin-only buttons for non-admins), shows the loading-screen transition
DatabaseStorage	JDBC connection factory; creates tables if missing; loads .env as a fallback to real env vars
UserService	Auth, registration, profile updates, password reset support
FacilservJRD	Facility CRUD, active/inactive toggling, hard-delete guard
BookServ	Booking CRUD, overlap detection, available-slot enumeration, "in use right now" check
OtpService / EmailService	Password-reset OTP generation, storage, verification, and delivery
LoadingSpinner	Reusable animated spinner (Lucide "loader" icon, rotated via cached bitmap)